Jasper OS Quantum Upgrade Method

CIP Number: JOSC-QUM-001

Status: FINAL — DRAFT FOR PATENT FILING | Version: 1.0.0 | Created: 2026-07-19

Authors: Jasper OS Security Architecture Team | Classification: Proprietary / Patent-Pending

---

DOCUMENT METADATA	
CIP Number	JOSC-QUM-001
Title	Jasper OS Quantum Upgrade Method — Master Specification
Status	**FINAL — DRAFT FOR PATENT FILING**
Version	1.0.0
Created	2026-07-19
Authors	Jasper OS Security Architecture Team
Classification	Proprietary / Patent-Pending
Supersedes	HYBRID_V1 CIP, PQ_ONLY CIP, PQ_NATIVE CIP, Aegis CIP, Chronos CIP
Keywords	post-quantum cryptography, recursive learning, air-gapped upgrade, DID governance, URIB, Aegis self-healing, Chronos orchestration, runtime enforcement

---

Table of Contents

Section 1 — Abstract

The **Jasper OS Quantum Upgrade Method (QUM)**, identified under CIP Number JOSC-QUM-001, discloses a novel, integrated framework for the

systematic, policy-governed transition of operating-system-level cryptographic primitives from classical asymmetric and symmetric algorithms to standardized post-quantum (PQ) alternatives across heterogeneous, distributed, and air-gapped computing environments. The method is structured as three sequential and formally gated migration phases — designated **HYBRID_V1**, **PQ_ONLY**, and **PQ_NATIVE** — through which an operating system progresses from dual-stack classical/PQ operation to full PQ-native cryptographic substrate status. Central to the invention is a **Recursive Learning Engine (RLE)** that applies federated, privacy-preserving machine learning to continuously optimize algorithm selection and migration pacing across heterogeneous node classes, informed by real-time telemetry, hardware capability fingerprints, and adversarial robustness criteria. The invention incorporates a deterministic **Air-Gapped Upgrade Pipeline (AGUP)** enabling verifiable, triple-signed cryptographic upgrade delivery to nodes operating without network connectivity, thereby extending the framework's applicability to the most security-sensitive deployment environments.

The QUM further discloses an **Aegis Self-Healing Subsystem** providing four-tier autonomous anomaly detection, cryptographic failure containment, and atomic rollback; a **Chronos Heartbeat Orchestration Engine** implementing epoch-driven, Byzantine-fault-tolerant lifecycle management across all upgrade events and key rotation operations; a **Decentralized Identity (DID) Governance Layer** providing cryptographically anchored, quorum-based policy management and phase-transition authorization consistent with the W3C DID Core Specification; a **Universal Resource Identity Bundle (URIB) Settlement Schema** constituting an immutable, Merkle-chained audit ledger of all cryptographic events; a **Runtime Enforcement Layer (REL)** enforcing algorithm policy at kernel and application levels; and an **Acceptance Criteria Validator (ACV)** providing deterministic, multi-category compliance gating at all phase transitions. Together, these subsystems constitute the first fully integrated, OS-native post-quantum cryptographic upgrade framework capable of eliminating quantum-threat windows during transition periods and achieving zero-downtime migration through autonomous, policy-driven orchestration across connected, distributed, and air-gapped environments.

Section 2 — Background and Motivation

2.1 Threat Landscape

The imminent practical realization of cryptographically relevant quantum computers poses an existential threat to the security foundations of modern computing infrastructure. Asymmetric cryptographic primitives upon which contemporary operating systems universally rely — specifically **RSA** (Rivest-Shamir-Adleman) and **ECC** (Elliptic Curve Cryptography) — are rendered computationally tractable by Shor's algorithm when executed on a sufficiently large fault-tolerant quantum processor. Current cryptographic security assumptions predicate RSA-2048 and ECC-P256 on the intractability of integer factorization and the elliptic curve discrete logarithm problem respectively, both of which Shor's algorithm solves in polynomial time on a quantum computer with sufficient qubit count and coherence.

The United States National Institute of Standards and Technology (NIST) completed its multi-year Post-Quantum Cryptography (PQC) standardization process, publishing three finalized Federal Information Processing Standards in 2024: **NIST FIPS 203** (ML-KEM, Module-Lattice-Based Key-Encapsulation Mechanism), **NIST FIPS 204** (ML-DSA, Module-Lattice-Based Digital Signature Algorithm), and **NIST FIPS 205** (SLH-DSA, Stateless Hash-Based Digital Signature Algorithm). These standards define the authoritative post-quantum primitive set for government and critical infrastructure deployment and form the cryptographic foundation of the present invention.

A particularly acute threat vector is the **harvest-now-decrypt-later (HNDL)** attack, in which an adversary with network access records ciphertext encrypted under classical primitives today, deferring decryption until a cryptographically relevant quantum computer becomes available. Encrypted data with long-lived sensitivity — including key material, identity credentials, and classified communications — is therefore already at risk, regardless of the current timeline for practical quantum computation. This threat makes the commencement of PQ migration an immediate operational imperative rather than a deferred engineering concern.

2.2 Problem Statement

Operating-system-level cryptographic migration presents a distinct class of engineering challenge beyond the replacement of individual application-layer algorithms. The cryptographic substrate of an operating system is deeply integrated with boot chains, kernel security modules, hardware security modules (HSMs), identity credential stores, inter-process communication protocols, and all dependent services. A naive "flag-day" migration — in which all classical primitives are simultaneously replaced by PQ equivalents — is operationally infeasible for production systems requiring continuous availability.

The specific challenges addressed by the present invention are: (1) execution of live cryptographic migration without service interruption across heterogeneous node classes with differing hardware capabilities and availability requirements; (2) safe and verifiable delivery of cryptographic upgrade artifacts to **air-gapped nodes** that have no network connectivity and cannot participate in live orchestration; (3) maintenance of coherent, auditable cryptographic identity governance across a distributed node population during a transition period in which nodes may simultaneously operate under different cryptographic configurations; and (4) autonomous optimization of migration pacing and algorithm selection in response to real-world telemetry without requiring manual per-node configuration.

2.3 Prior Art Gaps

Existing approaches to post-quantum cryptographic migration are characterized by one or more of the following deficiencies that the present invention is specifically designed to address:

- **Absence of Recursive Self-Optimizing Algorithm Selection:** Existing migration tooling applies static, pre-configured algorithm schedules that do not adapt to real-world node performance, error rates, or hardware capability variation. No existing system applies federated machine learning to continuously optimize primitive selection across a heterogeneous node population.
- **Lack of Unified Heartbeat Orchestration:** Prior art does not disclose a cryptographically signed, Byzantine-fault-tolerant epoch-driven orchestration engine capable of coordinating key rotation, policy synchronization, and

phase transition actions across an entire node population with tamper-evident audit trails.

- **Inadequate Air-Gapped Upgrade Chains:** No prior system discloses a deterministic, triple-signed, Merkle-verified upgrade pipeline capable of delivering cryptographic migration artifacts to air-gapped nodes with full chain-of-custody logging and SBOM traceability.
- **Absence of DID-Anchored Policy Governance:** Existing systems rely on centralized policy authorities vulnerable to single-point compromise. No prior art discloses the application of W3C-conformant decentralized identity governance with post-quantum signing keys to cryptographic upgrade policy management.
- **No URIB-Based Settlement Traceability:** Prior art lacks an immutable, Merkle-chained, cryptographically anchored audit ledger that records every upgrade event, epoch record, key rotation, and policy change with individual record provability via Merkle inclusion proofs.

2.4 Invention Scope

The Jasper OS Quantum Upgrade Method (QUM), disclosed in this Cryptographic Improvement Proposal, constitutes the first fully integrated, OS-native post-quantum cryptographic upgrade framework. The invention's scope encompasses the complete lifecycle of OS-level PQ migration: from initial dual-stack operation through classical primitive deprecation to full PQ-native steady state, together with all orchestration, governance, enforcement, learning, and audit subsystems required to execute that lifecycle autonomously, safely, and verifiably across connected, distributed, and air-gapped environments. The QUM is designed for deployment within the Jasper OS platform but is specified at a level of abstraction sufficient to inform implementation on any POSIX-compatible operating system kernel with appropriate HSM and TEE support.

Section 3 — Definitions and Terminology

The following terms are defined for use throughout this document. Each term is bolded on its first use in subsequent sections.

Term	Definition
CIP (Cryptographic Improvement Proposal)	A formal technical specification document that defines, governs, and records a discrete cryptographic architecture change within the Jasper OS ecosystem. CIPs serve as both engineering specifications and patent-filing artifacts.
PQ-Native / **PQ-Only** / **Hybrid-V1**	The three designated migration phases of the Jasper OS QUM. Hybrid-V1 denotes dual-stack classical/PQ operation; PQ-Only denotes exclusive PQ primitive enforcement with classical blacklisting; PQ-Native denotes full architectural integration of PQ cryptography as the default OS substrate with classical support entirely removed.
Aegis (Self-Healing Subsystem)	The autonomous anomaly detection, containment, and remediation engine within Jasper OS that monitors cryptographic primitive health, policy compliance, entropy integrity, and upgrade correctness, responding through a four-tier alert and response hierarchy.
Chronos (Heartbeat Orchestration Engine)	The epoch-driven lifecycle management engine that schedules, coordinates, and audits all cryptographic upgrade events, key rotation operations, policy synchronization actions, and cross-node coordination within the Jasper OS QUM, using a Byzantine-fault-tolerant distributed clock.
DID (Decentralized Identifier)	A globally unique, cryptographically verifiable identifier conformant with the W3C DID Core Specification v1.0. Within the Jasper OS QUM, DIDs serve as the identity anchors for nodes, governance authorities, signing keys,

Term	Definition
	and policy documents. The Jasper OS custom DID method is designated `did:jasper`.
URIB (Universal Resource Identity Bundle)	The immutable, cryptographically anchored, append-only audit ledger that records all upgrade events, epoch records, key rotations, policy changes, Aegis alerts, and settlement confirmations within the Jasper OS QUM as a Merkle-chained sequence of individually provable records.
Recursive Learning Engine (RLE)	The federated, privacy-preserving machine learning subsystem that continuously ingests telemetry signals from all nodes and produces algorithm recommendation vectors to optimize primitive selection and migration pacing across heterogeneous node classes, using secure multi-party computation for gradient aggregation.
Air-Gapped Upgrade Pipeline (AGUP)	The seven-stage deterministic process for compiling, signing, delivering, and applying cryptographic upgrade bundles to nodes operating without network connectivity, with full chain-of-custody logging and Merkle-verified integrity.
Runtime Enforcement Layer (REL)	The kernel-module and application-shim combination that intercepts all cryptographic syscalls and API invocations within Jasper OS, validates them against the current DID Governance policy anchor, and enforces one of three operational modes (PERMISSIVE, ENFORCING, STRICT) depending on the active migration phase.
Acceptance Criteria Validator (ACV)	The automated, deterministic compliance gating system that executes a six-category test suite before any phase transition, AGUP bundle application, or major policy change is permitted to proceed, producing PASS, CONDITIONAL, or FAIL outcomes with associated URIB records.
Algorithm Agility Matrix (AAM)	The runtime primitive switching logic component that maintains a policy-

Term	Definition
	governed registry of authorized cryptographic algorithms per node class and migration phase, enabling hot-swap of PQ primitive variants under RLE recommendation without requiring a full phase transition.
Migration Phase	One of three formally defined stages of the Jasper OS QUM: HYBRID_V1, PQ_ONLY, or PQ_NATIVE. Each phase has defined entry criteria, exit criteria, technical specifications, and rollback procedures, and is governed by DID Governance authorization and Chronos epoch management.
Heartbeat Epoch	The fundamental time unit of Chronos orchestration, representing a configurable time window (default: 30 minutes for server-class nodes) within which a defined set of scheduled actions are executed, recorded, and settled in the URIB. Each epoch is assigned a monotonically increasing uint64 identifier and is cryptographically signed by the Chronos master key.
Policy Anchor	A signed, DID-registered JSON-LD governance document that defines the currently authorized cryptographic algorithms, migration phase, quorum thresholds, and operational parameters for the Jasper OS QUM. Policy anchors are fetched by Chronos and Aegis every epoch and their hashes are stored in the URIB.
Cryptographic Primitive	An elementary cryptographic algorithm or operation, including key encapsulation mechanisms (KEMs), digital signature algorithms, hash functions, symmetric ciphers, and key derivation functions, upon which higher-order cryptographic protocols are constructed. In the Jasper OS QUM context, primitives are selected exclusively from the NIST PQC standardized set in PQ_ONLY and PQ_NATIVE phases.

Section 4 — System Architecture Overview

4.1 High-Level Architecture

The Jasper OS QUM is organized as a vertically layered architecture of eleven interdependent subsystems, each operating at a distinct level of abstraction from hardware root-of-trust to compliance gating. The layers are enumerated below in ascending order of abstraction:

Layer	Subsystem	Description and Primary Function
Layer 0	**Hardware Security Module (HSM) / Trusted Execution Environment (TEE)**	**Physical and firmware root-of-trust. Stores all key material in tamper-resistant hardware. Enforces algorithm whitelist at the microcode level. Provides certified True Random Number Generator (TRNG) entropy for all PQ key generation operations. Conforms to FIPS 140-3 Level 3 or higher.**
Layer 1	PQ-Native Cryptographic Primitive Library	Software implementation of all NIST PQC standardized primitives: ML-KEM-1024 (key encapsulation), ML-DSA-87 (digital signatures), and SLH-DSA-SHA2-256s (stateless hash-based signatures for long-lived certificates). All implementations are constant-time with documented side-channel mitigations. Interfaces with Layer 0 for key storage and entropy

Layer	Subsystem	Description and Primary Function
		sourcing.
Layer 2	**Algorithm Agility Matrix (AAM)**	**Runtime primitive switching logic that maintains a policy-governed registry of authorized algorithms per node class and per migration phase. Supports hot-swap of PQ primitive variants under RLE recommendation. Enforces blacklist restrictions during PQ_ONLY and PQ_NATIVE phases. Communicates authoritative algorithm selections to all higher layers.**
Layer 3	Recursive Learning Engine (RLE)	Federated, privacy-preserving machine learning subsystem for adaptive algorithm selection and migration pacing. Ingests telemetry from all subsystem layers. Produces algorithm recommendation vectors per node class. Uses Secure Multi-Party Computation (SMPC) for gradient aggregation. Delivers model weight updates via AGUP for air-gapped nodes.
Layer 4	**Chronos Heartbeat Orchestration Engine**	**Epoch-driven, Byzantine-fault-tolerant lifecycle management engine. Schedules and coordinates all upgrade events, key rotation, policy synchronization, RLE updates, and cross-node coordination**

Layer	Subsystem	Description and Primary Function
		actions. Signs each epoch record with the Chronos master key and anchors records in URIB. Manages missed-epoch detection and AGUP catchup bundle preparation.
Layer 5	Aegis Self-Healing Subsystem	Autonomous anomaly detection, containment, and remediation engine with four-tier alert hierarchy. Continuously monitors cryptographic primitive health, policy drift, entropy integrity, side-channel timing variance, and URIB anomalies. Executes automated responses including re-anchoring, node isolation, atomic rollback, and emergency quarantine.
Layer 6	**Air-Gapped Upgrade Pipeline (AGUP)**	**Seven-stage deterministic offline upgrade chain for nodes without network connectivity. Compiles and triple-signs upgrade bundles on connected build nodes. Delivers via encrypted physical media. Applies bundles atomically with Chronos local epoch driver integration and Aegis monitoring. Exports validation results to URIB on reconnection.**
Layer 7	DID Governance Layer	Cryptographically anchored, decentralized policy management using the custom `did:jasper` DID method. Manages

Layer	Subsystem	Description and Primary Function
		algorithm approvals, phase transition authorization, emergency rollback votes, and key revocation via quorum governance. Policy documents published as signed JSON-LD documents anchored in the DID Registry.
Layer 8	**URIB Settlement Schema**	**Immutable, append-only, Merkle-chained cryptographic audit ledger. Records all upgrade events, epoch records, key rotations, phase transitions, Aegis alerts, and policy changes. Supports DID-authenticated audit queries with Merkle inclusion proofs. Batch-settles per Chronos epoch with multi-anchor signing.**
Layer 9	Runtime Enforcement Layer (REL)	Kernel-module (JasperSecMod) and application-shim (JasperCryptAPI v3.0) combination that intercepts all cryptographic syscalls and API invocations, validates against current DID policy anchor, logs all events to URIB, and operates in PERMISSIVE, ENFORCING, or STRICT mode per active migration phase.
Layer 10	**Acceptance Criteria Validator (ACV)**	**Automated, deterministic six-category compliance gating system. Executes before any phase transition, AGUP bundle**

Layer	Subsystem	Description and Primary Function
		application, or major policy change. Categories cover cryptographic correctness, performance, Aegis integration, DID governance, URIB integrity, and air-gapped operations. Produces PASS / CONDITIONAL / FAIL outcomes logged to URIB.

4.2 Data Flow Summary

The following numbered sequence describes the canonical upgrade flow from initial policy trigger through execution, monitoring, and settlement:

1. **Policy Trigger:** A governance event (e.g., phase transition authorization, algorithm approval, or key rotation schedule) is approved by DID Governance Layer quorum vote and published as an updated Policy Anchor in the DID Registry.
2. **Chronos Epoch Scheduling:** Chronos reads the updated Policy Anchor at the next epoch boundary. It schedules the required actions (upgrade delivery, key rotation, RLE model update) across the node population for the subsequent epoch sequence.
3. **DID Resolution:** For each target node, Chronos resolves the node's DID Document to obtain current PQ public keys and service endpoints. Air-gapped nodes use locally cached DID snapshots delivered via AGUP.
4. **AGUP Bundle Preparation (Air-Gapped Nodes):** For nodes without network connectivity, the AGUP compiles a signed upgrade bundle containing all required artifacts, triple-signs it via DID-authorized keys, writes the Merkle root to URIB, and initiates physical media transfer.

5. **Connected Node Upgrade Delivery:** For networked nodes, Chronos delivers upgrade artifacts directly via encrypted, authenticated channels using ML-KEM-1024 key encapsulation and ML-DSA-87-signed payloads.
6. **ACV Pre-Flight Validation:** Before applying any upgrade, the ACV executes Category A (cryptographic correctness) and Category B (performance) pre-flight tests on the target node. A FAIL outcome blocks application and triggers Aegis Tier 3 alert.
7. **Atomic Upgrade Application:** The upgrade is applied atomically by Chronos local epoch driver. A rollback image of the prior state is preserved by Aegis. REL is updated to the new enforcement mode upon successful application.
8. **Aegis Real-Time Monitoring:** Throughout application, Aegis monitors all four detection mechanisms. Any anomaly at Tier 3 or above triggers immediate atomic rollback and URIB emergency entry.
9. **ACV Post-Flight Validation:** After application, ACV executes the full six-category suite. Results are signed by the node's DID key and stored locally pending URIB export.
10. **URIB Settlement:** All epoch events — epoch record, upgrade events, ACV results, Aegis alerts — are batched, Merkle-rooted, and settled in URIB with multi-anchor signing by the required quorum of URIB anchor nodes.
11. **RLE Telemetry Feedback:** Post-settlement, telemetry metrics (latency, error rates, Aegis scores, URIB settlement latency) are fed to the RLE as training signals. The RLE updates algorithm recommendation vectors for the subsequent epoch cycle.
12. **AAM Update:** RLE algorithm recommendations with confidence score ≥ 0.90 are applied to the AAM for the subsequent epoch. Recommendations below threshold trigger human-in-the-loop review via DID Governance dashboard.

4.3 Subsystem Interaction Matrix

Source Subsystem	Target Subsystem	Interaction Type	Protocol / Interface
DID Governance	Chronos	Policy Anchor delivery	Signed JSON-LD over DID service endpoint; ML-DSA-87 authenticated
DID Governance	Aegis	Policy Anchor refresh; Key revocation	Signed JSON-LD; revocation via DID Registry update
DID Governance	AGUP	Signing key authorization for bundles	DID Document key resolution; ML-DSA-87 triple-signing
Chronos	RLE	Epoch trigger for model weight update delivery	Internal IPC; epoch action scheduling
Chronos	AGUP	Bundle availability check; catchup bundle trigger	Internal IPC; epoch scheduling API
Chronos	URIB	Epoch record signing and settlement	ML-DSA-87 signed epoch record; URIB batch settlement API
Chronos	ACV	Compliance scan trigger per epoch	Internal IPC; epoch action scheduling
Aegis	RLE	Anomaly scores as negative training signals; rollback events	Internal telemetry bus
Aegis	URIB	Alert records; rollback entries	URIB event API; ML-DSA-87 signed records
Aegis	DID Governance	Tier 3/4 alert notification; emergency vote trigger	DID service endpoint push notification
RLE	AAM	Algorithm recommendation vector delivery	Internal API; confidence-gated recommendation
RLE	AGUP	Model weight bundle inclusion for air-gapped nodes	Signed model weight artifact in AGUP bundle
AGUP	URIB	Bundle Merkle root	URIB settlement

Source Subsystem	Target Subsystem	Interaction Type	Protocol / Interface
		anchoring; post-application ACV results	API; SLH-DSA-anchored Merkle root
REL	URIB	Enforcement event logging	Asynchronous URIB event write; ML-DSA-87 signed
REL	Chronos	Enforcement metrics for epoch health score	Internal telemetry bus
ACV	URIB	Test suite results logging	URIB event API; node DID key signed
ACV	Aegis	FAIL outcome triggers Tier 3 alert	Internal IPC; direct alert injection
HSM/TEE	PQ Primitive Library	Key storage, TRNG entropy, algorithm whitelist enforcement	FIPS 140-3 HSM API; PKCS#11 v3.0 extended for PQ
PQ Primitive Library	AAM	Primitive implementation availability reporting	Internal capability registration API

Section 5 — Migration Phase Specifications

Phase Overview

The Jasper OS QUM defines three sequential, formally gated migration phases. Each phase has defined entry criteria, exit criteria, rollback procedures, and technical specifications. Phase transitions require DID Governance quorum authorization and ACV PASS outcomes prior to commencement.

5.1 HYBRID_V1 Phase

5.1.1 Objective

The **HYBRID_V1** phase establishes dual-stack cryptographic operation, wherein classical and post-quantum algorithms execute concurrently for all protected operations. Classical algorithm fallback is guaranteed throughout this phase, ensuring zero-disruption migration for dependent services while PQ primitive performance and correctness are validated under production conditions. This phase constitutes the primary harvest-now-decrypt-later mitigation for data with long-lived sensitivity, as PQ key encapsulation is applied to all new key exchange operations from phase commencement.

5.1.2 Technical Specification

Parameter	Specification
Classical algorithm stack	RSA-4096 (key encapsulation, signatures); ECC-P384 (ECDH key exchange, ECDSA signatures)
Post-quantum algorithm stack	ML-KEM-768 (key encapsulation); ML-DSA-65 (digital signatures)
Key encapsulation method	Dual-encapsulation: independent classical (ECDH / RSA-OAEP) and PQ (ML-KEM-768) encapsulation executed in parallel; shared secrets combined via HKDF with XOR combiner; combined secret used for session key derivation
Digital signature verification	Dual-signature: both RSA/ECDSA and ML-DSA-65 signatures must independently verify for payload acceptance; either signature failure rejects payload
Certificate hierarchy	Hybrid X.509 v3 certificates with dual public key fields; PQ public key embedded via OID extension (draft-ietf-lamps-pq-composite-keys); both classical and PQ CA chains maintained
Protocol support	TLS 1.3 (RFC 8446) with hybrid key exchange per IETF draft-ietf-tls-hybrid-design; ECDH + ML-KEM-768 combined

Parameter	Specification
	key exchange in single ClientHello extension
Handshake overhead budget	< 15% additional latency relative to classical-only TLS 1.3 handshake on reference hardware (measured at p95 latency percentile)
Key material storage	Both classical and PQ key material stored in HSM under separate namespace partitions; classical keys in primary partition; PQ keys in PQ-designated partition with independent access controls
REL enforcement mode	PERMISSIVE — classical primitive invocations logged but not blocked; all invocations recorded in URIB

5.1.3 Entry Criteria

- Policy trigger approved by DID Governance Layer with ≥ 2/3 quorum vote authorizing HYBRID_V1 commencement.
- ACV full suite PASS outcome on all target nodes prior to phase commencement.
- Chronos Epoch E0 formally commenced and anchored in URIB.
- HSM firmware supporting dual-namespace key storage validated and installed on all target nodes.
- JasperCryptAPI v3.0 hybrid-mode shim deployed to all target application environments.

5.1.4 Exit Criteria

- RLE confidence score ≥ 0.92 for PQ primitives (ML-KEM-768, ML-DSA-65) across all monitored endpoints, sustained for ≥ 5 consecutive Chronos epochs.
- Zero Aegis Tier 3 or Tier 4 alerts across all nodes for 30 consecutive heartbeat epochs.

- URIB settlement confirmation for 100% of HYBRID_V1 transactions; zero unsettled records older than 2 epochs.
- ACV Category A (cryptographic correctness) and Category E (URIB integrity) PASS on full node population.
- DID Governance quorum vote (≥ 2/3) authorizing advancement to PQ_ONLY phase.

5.1.5 Rollback Procedure

In the event of a Tier 3 or Tier 4 Aegis alert during HYBRID_V1 operation, Aegis executes an atomic rollback to classical-only operation within one heartbeat epoch. The rollback procedure: (1) Aegis disables PQ primitive invocations in AAM; (2) REL PERMISSIVE mode logs classical-only operation as a policy event; (3) a URIB ROLLBACK record is created with full Aegis diagnostic payload; (4) RLE receives the rollback event as a weighted negative training signal; (5) DID Governance dashboard is notified for human review and authorization of PQ re-enablement. Rollback to classical-only does not constitute a migration phase regression and does not require DID Governance quorum vote.

5.2 PQ_ONLY Phase

5.2.1 Objective

The **PQ_ONLY** phase deprecates all classical cryptographic primitives from active operation across all system layers. PQ algorithms become the exclusive permitted cryptographic substrate. Classical algorithm invocations are actively blocked at the kernel level by the REL ENFORCING mode, raising a `SecurityException` and generating an Aegis alert. This phase achieves full forward-secrecy elimination of quantum-vulnerable key material and establishes the operational baseline for PQ_NATIVE migration.

5.2.2 Technical Specification

Parameter	Specification
Key encapsulation	ML-KEM-1024 exclusively; all ECDH and RSA-OAEP invocations blacklisted

Parameter	Specification
Digital signatures	ML-DSA-87 exclusively for all transient signing operations; SLH-DSA-SHA2-256s for long-lived certificate anchor signatures
Classical primitive blacklist	Enforced at REL JasperSecMod kernel module level; RSA, ECC, DSA, DH invocations return `SecurityException` and trigger Aegis Tier 2 alert minimum; repeated violations escalate to Tier 3
Certificate migration	All X.509 certificates reissued as pure PQ certificates (ML-DSA-87 CA signatures, SLH-DSA-SHA2-256s root); legacy hybrid and classical certificates revoked via CRL and OCSP-PQ; all revocations logged in URIB
ML-KEM key rotation schedule	ML-KEM-1024 encapsulation keypairs rotated every 90 Chronos heartbeat epochs (default: 45 hours for server-class nodes)
ML-DSA signing key rotation schedule	ML-DSA-87 signing keys rotated every 180 Chronos heartbeat epochs (default: 90 hours for server-class nodes)
Entropy requirements	Minimum 512-bit entropy from hardware TRNG for all PQ key generation; TRNG health continuously monitored per NIST SP 800-90B; Aegis Tier 2 alert on entropy source degradation
REL enforcement mode	ENFORCING — classical primitive invocations blocked; `SecurityException` raised and logged to URIB

5.2.3 Entry Criteria

- All HYBRID_V1 exit criteria verified and attested in URIB.
- DID Governance policy anchor updated to PQ_ONLY mode with ≥ 2/3 quorum authorization.
- Chronos Epoch E1 formally commenced and anchored in URIB.
- All X.509 certificates reissued as pure PQ certificates or formally scheduled for reissuance within the first 10 epochs of PQ_ONLY phase.

- REL ENFORCING mode validated on all target nodes via ACV Category A pre-flight.

5.2.4 Exit Criteria

- 100% of system cryptographic primitives confirmed PQ-native by ACV full scan across all nodes.
- URIB settlement ledger shows zero classical primitive invocation records for 60 consecutive epochs (verified by REL log aggregation).
- RLE confidence score ≥ 0.97 for ML-KEM-1024 and ML-DSA-87 across all node classes.
- ACV PASS on all six categories across all nodes.
- DID Governance quorum vote (≥ 2/3) authorizing advancement to PQ_NATIVE phase.

5.2.5 Regression Handling

No rollback from PQ_ONLY to HYBRID_V1 is permissible under normal operational parameters. An emergency regression to HYBRID_V1 is permitted only upon satisfaction of all of the following conditions simultaneously: (1) a concurrent Tier 4 Aegis emergency event is active affecting ≥ 25% of the production node population; (2) a DID Governance emergency vote achieves ≥ 2/3 quorum of all authorized governance DID holders within one epoch of the triggering event; (3) the regression authorization is recorded in URIB with all governance DID signatures attached. Emergency regression triggers automatic incident reporting and mandatory post-incident review within 5 epochs of regression commencement.

5.3 PQ_NATIVE Phase

5.3.1 Objective

The **PQ_NATIVE** phase constitutes the terminal, steady-state cryptographic architecture of Jasper OS. Post-quantum cryptography is fully integrated as the native, default OS cryptographic substrate. Classical algorithm support is entirely removed from the codebase, HSM firmware, and hardware configuration. This phase is not a transitional state but a permanent operational baseline subject to ongoing

RLE optimization, Chronos orchestration, and lightweight CIP amendment for future PQ algorithm variant updates.

5.3.2 Technical Specification

Parameter	Specification
Kernel cryptographic API	JasperCryptAPI v3.0 — PQ-only interface; all legacy crypto API endpoints (RSA, ECC, DSA, DH) removed from compiled kernel; invocation raises `ENOTSUP` at syscall level
HSM firmware	PQ-Native HSM firmware flashed; classical algorithm accelerators (RSA engine, ECC engine) disabled in microcode via write-once OTP fuse; PQ accelerators (lattice NTT units) activated
Algorithm Agility Matrix	AAM locked to PQ-only primitives; inter-variant agility retained (ML-KEM-768 ↔ ML-KEM-1024 security level switching; ML-DSA-65 ↔ ML-DSA-87 signing key size switching) under RLE recommendation and DID Governance policy
OS boot chain	PQ-signed bootloader verified via ML-DSA-87 signature; UEFI Secure Boot root certificate replaced with SLH-DSA-SHA2-256s self-signed root; full boot chain Merkle-hashed and anchored in URIB at each boot
Side-channel mitigations	Constant-time ML-KEM and ML-DSA implementations verified via compiler instrumentation (clang -ftime-trace analysis); cache-flush routines (CLFLUSH) executed between sequential cryptographic operations on shared hardware; Aegis timing variance monitoring active
Entropy pooling	Continuous hardware TRNG seeding of OS entropy pool; TRNG health monitoring per NIST SP 800-90B; secondary entropy mixing via DRBG (CTR_DRBG with AES-256) as conditioner; minimum pool entropy floor enforced by Aegis

Parameter	Specification
REL enforcement mode	STRICT — all non-whitelisted PQ primitive variants blocked; policy immutable until next DID Governance authorized update; zero exceptions without quorum vote

5.3.3 Entry Criteria

- PQ_ONLY phase fully settled in URIB with all exit criteria attestations present.
- DID Governance policy anchor updated to PQ_NATIVE mode with ≥ 2/3 quorum authorization.
- PQ-Native HSM firmware validated and staged for all target nodes via AGUP.
- JasperCryptAPI v3.0 PQ-Native build validated on all target nodes by ACV full suite PASS.
- All dependent services validated for PQ_NATIVE compatibility by ACV Category A and Category D scans.

5.3.4 Exit Criteria

PQ_NATIVE is the terminal, steady-state phase. There are no exit criteria for normal operations. Ongoing maintenance — including key rotation, RLE model updates, Chronos epoch management, and Aegis monitoring — continues indefinitely under the established governance framework. Future algorithm transitions (e.g., adoption of NIST PQC Round 5+ standardized algorithms) are governed by lightweight CIP amendment process rather than full phase transition.

5.3.5 Future Algorithm Agility

The RLE continuously monitors NIST PQC Round 5 and subsequent candidate algorithms for standardization signals. Upon standardization, a lightweight CIP amendment is prepared and submitted to DID Governance for quorum approval. Upon approval, the new algorithm is registered in the AAM and the PQ-Native Cryptographic Primitive Library. Hot-swap of PQ algorithm variants within the AAM does not constitute a phase transition and does not require ACV full suite execution, but does require ACV Category A (cryptographic correctness) validation and URIB settlement of the AAM update event.

Section 6 — Recursive Learning Engine (RLE)

6.1 Purpose

The **Recursive Learning Engine (RLE)** provides autonomous, telemetry-driven optimization of cryptographic algorithm selection and migration pacing across all system nodes. It eliminates the need for manual per-node migration configuration by continuously learning from real-world performance, error, and anomaly signals and translating that learning into actionable, confidence-scored algorithm recommendations for the Algorithm Agility Matrix.

6.2 Architecture

6.2.1 Input Signals

Signal Category	Description
Latency metrics	Per-primitive, per-node-class encapsulation/decapsulation and sign/verify latency at p50, p95, p99 percentiles per epoch
Error rates	Cryptographic operation failure rates; `SecurityException` frequency per REL enforcement mode
Aegis anomaly scores	Normalized Tier 1–4 alert frequency and severity per node and epoch
URIB settlement latency	Time from event generation to anchored URIB settlement, per event type
Chronos epoch health scores	Per-epoch health score (0.0–1.0) reported by Chronos across the node population
Hardware capability fingerprints	CPU generation, HSM model, available PQ hardware accelerators, available TRNG throughput — reported at node enrollment and on HSM firmware update

6.2.2 Model Architecture

The RLE employs a federated, privacy-preserving learning model in which raw telemetry never leaves the originating node. Each node trains a local model replica on its own telemetry and computes a local gradient update. Gradient aggregation across the node population is performed via **Secure Multi-Party Computation (SMPC)**, producing a global model update without exposing individual node telemetry to any aggregating party. The global model produces an **algorithm recommendation vector** per node class (server, edge, air-gapped, embedded), specifying the recommended PQ algorithm variant and security parameter level for each cryptographic operation type.

6.2.3 Operational Parameters

- **Update frequency:** Model weights updated every 10 Chronos heartbeat epochs.
- **Confidence threshold:** Recommendations acted upon automatically only when confidence score ≥ 0.90. Scores below threshold trigger human-in-the-loop review via DID Governance dashboard and require governance acknowledgment before AAM update.
- **Model output scope:** Recommendations cover: preferred KEM security level (ML-KEM-768 vs. ML-KEM-1024), preferred signature security level (ML-DSA-65 vs. ML-DSA-87), key rotation epoch interval recommendations, and migration phase readiness score per node class.

6.3 Feedback Loop

The RLE operates within a closed-loop feedback cycle with the following sequence per epoch:

13. **Telemetry Collection:** Each node generates telemetry from all active subsystem layers (REL, Aegis, Chronos, URIB settlement) and stores locally.
14. **Local Model Training:** Node trains local model replica on accumulated telemetry; computes gradient update.

15. **SMPC Gradient Aggregation:** Gradient updates from all participating nodes aggregated via SMPC protocol every 10 epochs; global model weights updated.
16. **AAM Recommendation Delivery:** RLE delivers updated algorithm recommendation vector to AAM with per-recommendation confidence score.
17. **Chronos Scheduling:** Chronos schedules any approved AAM updates for the subsequent epoch sequence.
18. **AGUP Integration:** For air-gapped nodes, updated model weights included in next AGUP bundle.
19. **Upgrade Execution:** Chronos epoch driver applies AAM updates; AGUP applies bundles to air-gapped nodes.
20. **Aegis Monitoring:** Aegis monitors post-update operation for anomalies; any Tier 2+ event generates a negative training signal.
21. **URIB Settlement:** All RLE-driven AAM updates and their outcomes recorded in URIB with full telemetry summary.
22. **Signal Return:** Post-settlement outcomes (success, anomaly, rollback) returned to RLE as weighted training signals for the next cycle.

6.4 Adversarial Robustness

The RLE implements **Byzantine-fault-tolerant gradient aggregation**: the SMPC protocol applies a trimmed-mean or Krum aggregation rule that excludes gradient updates from nodes whose contributions deviate statistically from the population distribution by more than a configurable threshold (default: 2.5 standard deviations). Outlier node telemetry is quarantined by Aegis — flagged nodes are excluded from SMPC gradient rounds until Aegis Tier 1 or above clears — preventing poisoning attacks from compromised or malfunctioning nodes from corrupting the global model. All gradient aggregation sessions are recorded in URIB with participant node DID attestations.

6.5 Air-Gapped RLE Operation

For air-gapped nodes, the full federated SMPC protocol is not available due to the absence of network connectivity. The air-gapped RLE operating mode functions as follows: updated global model weights — approved by DID Governance and signed by the RLE master key (ML-DSA-87) — are included in each AGUP bundle as a signed artifact. The air-gapped node applies the new model weights locally upon AGUP bundle ingestion and performs local inference only against its own telemetry. Local algorithm recommendations are generated without SMPC participation. Telemetry is batched locally and exported via the **Secure Physical Media Transfer Protocol (SPMTP)** on the next available connection window or physical media exchange, at which point the node's telemetry is incorporated into the next SMPC aggregation round.

Section 7 — Air-Gapped Upgrade Pipeline (AGUP)

7.1 Purpose

The **Air-Gapped Upgrade Pipeline (AGUP)** provides a deterministic, verifiable, and fully auditable mechanism for delivering cryptographic upgrade artifacts — including firmware deltas, PQ primitive library updates, RLE model weights, URIB settlement anchors, Chronos epoch configuration, DID policy updates, and ACV test suites — to nodes operating without network connectivity. The AGUP extends the full guarantees of the Jasper OS QUM to the most security-sensitive deployment environments where network isolation is a mandatory operational requirement.

7.2 Pipeline Architecture

The AGUP is structured as seven sequential, formally defined stages. Each stage produces a documented artifact and a URIB record:

Stage	Name	Specification
Stage 1	Upgrade Bundle	Executed on a connected,

Stage	Name	Specification
	Compilation	HSM-equipped build node. Bundle contents: signed firmware deltas (delta-compressed against prior version hash); updated RLE model weights (ML-DSA-87 signed); new URIB settlement anchors; Chronos epoch configuration for the target node class; updated DID policy snapshot; ACV test suite version current to the build epoch. All components inventoried in SBOM per NTIA minimum elements.
Stage 2	Bundle Signing	Bundle signed independently by three DID-authorized signing authorities using ML-DSA-87 governance keys. Signing requires concurrent availability of three distinct governance DID holders (quorum subset of DID Governance Layer). A Merkle tree is constructed over all bundle components; the Merkle root is recorded in the URIB as a SETTLEMENT event anchored by SLH-DSA-SHA2-256s before bundle release.
Stage 3	Integrity Manifest	A comprehensive Software Bill of Materials (SBOM) is generated per NTIA Minimum Elements for a Software Bill of Materials (June 2021). The SBOM enumerates all components with supplier, version, hash (SHA3-512), and

Stage	Name	Specification
		dependency relationships. The SBOM is embedded in the bundle as a standalone artifact and signed with SLH-DSA-SHA2-256s to provide long-term signature validity independent of signing key rotation schedules.
Stage 4	Physical Transfer	The compiled bundle is written to encrypted removable media. Media encryption uses the target node's current ML-KEM-1024 encapsulation public key (from its locally cached DID Document), ensuring only the target node can decapsulate the media encryption key. Chain-of-custody is logged: each custodian signs a transfer record with their personal DID key (ML-DSA-87); all transfer records attached to the URIB bundle record.
Stage 5	Air-Gapped Node Ingestion	Upon physical media connection to the air-gapped node: (a) the node decapsulates the media encryption key using its HSM-resident ML-KEM-1024 private key; (b) the node verifies all three ML-DSA-87 bundle signatures against its locally cached DID Document for each signing authority; (c) the node verifies the Merkle root of all bundle components against the embedded manifest; (d) ACV pre-flight suite (Category A and Category C) is executed on the

Stage	Name	Specification
		bundle artifacts prior to application. Any verification failure aborts ingestion and generates a local Aegis alert logged to the local URIB shard.
Stage 6	Staged Application	Chronos local epoch driver applies the bundle artifacts in a defined atomic transaction sequence: firmware delta first, then PQ library update, then Chronos epoch configuration, then RLE model weights, then DID policy snapshot, then ACV test suite update. At each step, Aegis monitors for anomalies. If any Aegis Tier 3 event occurs during application, the transaction is aborted and the node atomically rolls back to the pre-application state (preserved as a signed rollback snapshot). The rollback snapshot is stored in the local URIB shard and exported on reconnection.
Stage 7	Post-Application Validation	Full ACV six-category test suite executed post-application. Results signed by the node's DID key (ML-DSA-87) and stored in the local URIB shard. Upon the next available connection window or physical media exchange opportunity, the ACV results, local URIB shard delta, and application status report are exported and merged into the master URIB per the merge protocol defined in Section 11.6.

7.3 Determinism Guarantee

A foundational AGUP design requirement is determinism: an identical bundle applied to a node in an identical pre-application state must produce an identical post-application state. This property is verified by computing a SHA3-512 state hash of the node's cryptographic configuration (active key fingerprints, AAM state, Chronos epoch position, REL enforcement mode, and HSM firmware version) before and after bundle application, and comparing the post-application hash against the expected output hash embedded in the bundle manifest and signed by the build node. State hash mismatch after application triggers Aegis Tier 3 alert and automatic rollback. Determinism verification results are included in the Stage 7 ACV report.

7.4 Latency Budget

Stages 5 and 6 of the AGUP pipeline (ingestion and staged application) must complete within one local Chronos heartbeat epoch. For server-class air-gapped nodes, the default epoch duration is 30 minutes. For edge and embedded node classes, the epoch duration is configurable and extended appropriately (default: 4 hours for edge nodes). If Stages 5–6 cannot be completed within one epoch, Chronos local epoch driver pauses the epoch clock and resumes from the interruption point upon operator reauthorization via a signed DID governance key held by the on-site administrator. This exception is logged as an epoch extension event in the local URIB shard.

Section 8 — Aegis Self-Healing Subsystem

8.1 Purpose

The **Aegis Self-Healing Subsystem** provides autonomous detection, containment, and remediation of cryptographic failures, policy violations, upgrade anomalies, and entropy degradation events. Aegis operates continuously across all migration phases and is the primary safety net ensuring that cryptographic failures

are detected and contained before they can propagate to dependent services or compromise the integrity of the URIB audit record.

8.2 Detection Mechanisms

- **Cryptographic Primitive Health Monitoring:** Continuous validation of active key material integrity via challenge-response micro-probes executed every epoch against HSM-resident key pairs. A probe failure (failed signature verification or decapsulation) triggers immediate Tier 3 escalation.
- **Policy Drift Detection:** Real-time comparison of the node's active cryptographic configuration (AAM state, REL enforcement mode, active algorithm set) against the current DID Governance Policy Anchor. Any discrepancy triggers Tier 2 automated re-anchoring.
- **Entropy Health Monitoring:** Continuous TRNG health testing per NIST SP 800-90B, including startup tests (repetition count test, adaptive proportion test) and on-demand tests executed every epoch. Entropy source degradation triggers Tier 2 alert and DRBG fallback activation.
- **Side-Channel Telemetry:** Timing variance monitoring for constant-time implementation correctness: statistical analysis of cryptographic operation timing distributions; deviations exceeding 3 standard deviations from the baseline constant-time profile trigger Tier 2 alert and RLE notification for further analysis.
- **URIB Anomaly Detection:** Monitoring of URIB settlement patterns for unexpected gaps (missing epoch records), out-of-order predecessor hash chains, or settlement latency exceeding 2× the epoch duration baseline. Any detected anomaly triggers Tier 2 alert and manual URIB integrity audit scheduling.

8.3 Response Tiers

Tier	Trigger Condition	Automated Response	Human Notification
Tier 1 (Advisory)	Minor latency deviation (>10% above p99 baseline); non-critical telemetry anomaly	Log event to URIB; forward anomaly score to RLE as telemetry signal; no operational change	No immediate notification; aggregate in Chronos epoch health report
Tier 2 (Warning)	Policy drift detected; entropy degradation; timing variance anomaly; URIB settlement gap	Re-anchor node to current DID Policy Anchor; activate DRBG fallback if entropy-triggered; log to URIB; update Chronos epoch health score	Governance dashboard alert; on-call engineer notification via DID service endpoint
Tier 3 (Critical)	Key material corruption detected; challenge-response probe failure; ACV Category A FAIL; AGUP Merkle verification failure; state hash mismatch post-AGUP	Isolate affected node from cryptographic operations; trigger atomic rollback to last-known-good state; create URIB ROLLBACK record; suspend node from Chronos epoch participation pending review	Immediate notification to all DID Governance authorized nodes; incident ticket auto-created; RLE receives weighted negative training signal
Tier 4 (Emergency)	Systemic PQ primitive failure affecting ≥ 25% of node population; DID Registry compromise indicators; URIB chain integrity failure; HSM firmware integrity violation	Initiate full node population quarantine of affected segment; create URIB EMERGENCY entry with full diagnostic payload; suspend all cryptographic operations on affected nodes pending manual review; freeze Chronos epoch progression for affected node class	All authorized DID holders notified via all available service endpoints; escalation to Jasper OS Security Architecture Team on-call; mandatory incident command structure activation

8.4 Rollback Mechanism

At each AGUP bundle application and at each Chronos-scheduled upgrade event, Aegis preserves a cryptographically signed snapshot of the pre-application node state. The snapshot is stored in the HSM-resident rollback partition and includes: active key fingerprints, AAM state hash, REL enforcement mode, Chronos epoch position, DID policy anchor hash, and PQ library version hash. Upon rollback trigger, Aegis atomically restores the node to the snapshot state in a single transaction. Post-rollback, a new URIB ROLLBACK record is created containing the rollback trigger event, snapshot hash, and restoring Aegis version. RLE receives the rollback event as a weighted negative training signal, which reduces the confidence score assigned to the upgrade artifact type that triggered the rollback. Chronos resumes epoch participation for the rolled-back node after Aegis confirms restoration state integrity via a fresh challenge-response probe.

8.5 Self-Healing Loop

23. **Detection:** Aegis identifies anomaly via one of five detection mechanisms.
24. **Tier Classification:** Anomaly scored and assigned to Tier 1–4 based on severity and scope.
25. **Automated Remediation:** Tier-appropriate automated response executed (logging, re-anchoring, rollback, quarantine).
26. **ACV Validation:** Post-remediation ACV Category A and Category C (minimum) executed to confirm restored integrity.
27. **URIB Settlement:** Full remediation event (anomaly, response, ACV result) settled in URIB as a linked record sequence.
28. **Chronos Epoch Resume:** If epoch was paused during Tier 3/4 response, Chronos resumes upon Aegis clearance and ACV PASS.
29. **RLE Feedback:** Aegis scores and remediation outcomes forwarded to RLE as training signals for the next model update cycle.

Section 9 — Chronos Heartbeat Orchestration Engine

9.1 Purpose

The **Chronos Heartbeat Orchestration Engine** provides deterministic, epoch-driven lifecycle management of all cryptographic upgrade events, key rotation operations, policy synchronization actions, RLE model delivery, and cross-node coordination within the Jasper OS QUM. Chronos ensures that all upgrade activities occur in a predictable, auditable, and Byzantine-fault-tolerant schedule, with each epoch constituting a tamper-evident unit of work anchored in the URIB.

9.2 Epoch Architecture

Epoch Field	Specification
Epoch unit (server-class)	30-minute configurable time window
Epoch unit (edge-class)	4-hour configurable time window
Epoch unit (air-gapped)	Manual trigger or local operator-authorized timer
Epoch ID	Monotonically increasing unsigned 64-bit integer (uint64); globally unique per node cluster
Start timestamp	International Atomic Time (TAI) clock; ISO 8601 extended format with nanosecond precision
End timestamp	TAI ISO 8601; recorded upon epoch completion or timeout
Node class mask	Bitmask indicating which node classes this epoch record applies to
Scheduled actions list	Ordered list of action identifiers with expected completion timestamps
Health score	Float in range [0.0–1.0]; computed from REL enforcement metrics, Aegis alert frequency, and ACV compliance rate across the epoch
Aegis status flag	Enum: CLEAR / TIER1 / TIER2 / TIER3 / TIER4 — highest Aegis tier active during epoch

Epoch Field	Specification
Epoch record signature	ML-DSA-87 signature by Chronos master key over all epoch record fields
URIB anchor	Each epoch record anchored in URIB as an EPOCH event type; Merkle-chained with predecessor

9.3 Scheduled Actions per Epoch

- **Key rotation check:** Compare active key ages against rotation schedule; initiate rotation for any key past scheduled epoch interval.
- **RLE model weight update delivery:** If a new RLE model weight update is available and confidence-approved, deliver to AAM for application in the current epoch.
- **AGUP bundle availability check:** For any air-gapped nodes in the managed cluster, check for completed bundle preparation and initiate transfer logistics notification.
- **ACV compliance scan:** Trigger ACV Category E (URIB integrity) and Category D (DID governance) scans as routine maintenance.
- **DID policy anchor refresh:** Fetch and verify current DID Policy Anchor from DID Registry; compare hash against URIB-recorded previous anchor; update Aegis and REL if changed.
- **URIB settlement reconciliation:** Verify all events from the previous epoch are settled in URIB; identify any gaps and trigger Aegis Tier 1 advisory for investigation.
- **Aegis health report:** Aggregate Aegis detection metrics for the epoch into the epoch health score; export to RLE telemetry bus.

9.4 Cross-Node Synchronization

Chronos implements a **Byzantine-fault-tolerant distributed epoch clock** based on a variant of Practical Byzantine Fault Tolerance (PBFT). All connected nodes in a Chronos cluster participate in epoch consensus: a minimum of $n \geq 3f+1$ total nodes

are required to tolerate up to f Byzantine faulty nodes. Each epoch is confirmed only when ≥ 2f+1 nodes (a strict majority in a Byzantine fault model) have agreed on the epoch start timestamp, epoch ID, and scheduled action set. Phase-transition actions — including HYBRID_V1 → PQ_ONLY and PQ_ONLY → PQ_NATIVE transitions — require epoch consensus before commencement. No phase-transition action is executed on any node in the cluster until the consensus quorum is achieved.

9.5 Missed Epoch Handling

A node that misses more than 3 consecutive Chronos epochs (due to downtime, network partition, or AGUP delay) is flagged by Aegis with a Tier 1 advisory. After 5 consecutive missed epochs, the node is flagged Tier 2 and excluded from phase-transition epoch consensus participation. After 10 consecutive missed epochs, the node is considered epoch-lapsed and removed from the active node cluster roster pending an AGUP catchup bundle. The catchup bundle is prepared by the AGUP pipeline and contains all epoch actions missed during the lapse period, compressed and ordered chronologically. No phase-transition actions are permitted for an epoch-lapsed node until it is epoch-current and has passed a fresh ACV Category C (Aegis integration) and Category D (DID governance) scan.

9.6 Epoch Audit Trail

Every epoch record — including epoch ID, start/end timestamps, node class mask, scheduled actions list, completed actions list, health score, Aegis status flag, and Chronos master key signature — is permanently stored in the URIB as an EPOCH event. Epoch records are Merkle-chained: each record's predecessor hash field contains the SHA3-512 hash of the immediately preceding epoch record in the URIB, forming a tamper-evident chain analogous to a blockchain. The Merkle root of each epoch batch is independently signed by ≥ 2/3 of URIB anchor nodes before the batch is committed. Tampering with any single epoch record invalidates the predecessor hash chain from that point forward, providing immediate detectability.

Section 10 — Decentralized Identity (DID) Governance

10.1 Purpose

The **DID Governance Layer** provides cryptographically anchored, decentralized governance of all upgrade policies, algorithm approvals, phase transition authorizations, key revocation actions, and emergency responses within the Jasper OS QUM. By grounding all governance actions in W3C-conformant Decentralized Identifiers with post-quantum signing keys, the QUM eliminates centralized governance authorities as single points of compromise while maintaining a verifiable, auditable governance record anchored in the URIB.

10.2 DID Framework

Framework Element	Specification
DID method	`did:jasper` — custom DID method registered per W3C DID Core Specification 1.0; method specification published as a companion document to this CIP
DID Document: signing keys	ML-DSA-87 verification method for all governance signing operations; key ID format: `did:jasper:[node-id]#key-ml-dsa-[seq]`
DID Document: encryption keys	ML-KEM-1024 key agreement method for encrypted communications; key ID format: `did:jasper:[node-id]#key-ml-kem-[seq]`
DID Document: service endpoints	Governance dashboard endpoint; URIB query endpoint; Chronos epoch API endpoint; Aegis alert notification endpoint
DID Document: key rotation history	All prior verification method key IDs retained with revocation timestamps; verifiable via DID Document history API
DID Document: delegation rules	Explicit delegation chains for multi-tier governance authorization (e.g., governance key → signing key delegation for automated Chronos

Framework Element	Specification
	epoch signing)
DID Registry	Distributed ledger anchored to Jasper OS cluster consensus layer; Byzantine-fault-tolerant with n ≥ 3f+1 registry nodes; air-gapped nodes use locally cached DID Document snapshots updated via AGUP; snapshot staleness beyond 5 epochs triggers Aegis Tier 1 advisory

10.3 Governance Actions Requiring DID Authorization

Governance Action	Required Quorum	DID Key Type	URIB Event Type
Algorithm approval (new PQ primitive or variant)	≥ 2/3 of authorized governance DID holders	ML-DSA-87 governance key	POLICY_CHANGE
Phase transition trigger (any phase)	≥ 2/3 of authorized governance DID holders	ML-DSA-87 governance key	PHASE_TRANSITIO N
Emergency rollback from PQ_ONLY to HYBRID_V1	≥ 2/3 + concurrent Tier 4 Aegis event required	ML-DSA-87 emergency key (separate key purpose)	ROLLBACK + AEGIS_ALERT
URIB audit access (read-only query)	≥ 1/2 of authorized audit DID holders	ML-DSA-87 audit key	N/A (query log)
RLE model update approval	≥ 1/2 of authorized governance DID holders	ML-DSA-87 governance key	POLICY_CHANGE
New node enrollment into cluster	≥ 1/3 of authorized governance DID holders	ML-KEM-1024 enrollment key (for encrypted enrollment credential)	SETTLEMENT
DID key revocation and replacement	≥ 2/3 of authorized governance DID holders	ML-DSA-87 governance key	POLICY_CHANGE

Governance Action	Required Quorum	DID Key Type	URIB Event Type
CIP amendment approval	≥ 2/3 of authorized governance DID holders	ML-DSA-87 governance key	POLICY_CHANGE

10.4 Policy Anchor

Governance policies are published as **signed JSON-LD documents** anchored in the DID Registry. Each Policy Anchor document specifies: the currently authorized cryptographic algorithm set per operation type; the active migration phase; quorum thresholds per action type; RLE confidence thresholds; Chronos epoch duration configuration per node class; and the URIB retention policy hash. Policy Anchor documents are signed by the issuing governance DID holder's ML-DSA-87 key, with the signature verified by all consuming subsystems (Chronos, Aegis, REL, AAM). The SHA3-512 hash of the active Policy Anchor is stored in every URIB EPOCH record, creating an epoch-by-epoch audit trail of policy evolution.

10.5 Key Compromise Handling

Upon detection or notification of a DID key compromise: (1) the compromised key is immediately revoked via a governance quorum vote; (2) the revocation is published in the DID Registry as a DID Document update with the compromised key marked as revoked with a revocation timestamp; (3) Chronos propagates the revocation to all connected nodes in the next epoch via Policy Anchor refresh; (4) Aegis suspends all cryptographic operations authenticated by the compromised key on all affected nodes until a new DID Document with replacement keys is confirmed by each node; (5) AGUP prepares a key revocation bundle for all air-gapped nodes containing the updated DID Document snapshot and Chronos epoch configuration; (6) all URIB records signed by the compromised key are flagged with a revocation annotation but not deleted, preserving audit trail integrity while marking the affected records for enhanced scrutiny.

Section 11 — URIB Settlement Schema

11.1 Purpose

The **Universal Resource Identity Bundle (URIB) Settlement Schema** constitutes the immutable, cryptographically auditable record of all upgrade events, cryptographic operations, policy changes, epoch records, and system state transitions within the Jasper OS QUM. The URIB provides the authoritative audit trail required for patent filing, regulatory compliance, security incident investigation, and governance accountability. Its design ensures that no record can be modified, deleted, or reordered without immediate detectability via the Merkle-chained predecessor hash structure.

11.2 URIB Record Schema

Field Name	Type	Description
`urib_id`	UUID v7	Time-ordered universally unique record identifier; millisecond precision timestamp embedded in UUID structure per RFC 9562
`event_type`	Enum	One of: EPOCH, UPGRADE, KEY_ROTATION, PHASE_TRANSITION, AEGIS_ALERT, POLICY_CHANGE, SETTLEMENT, ROLLBACK
`node_id`	DID string	Fully qualified `did:jasper` DID of the originating node
`epoch_id`	uint64	Chronos epoch ID during which this event occurred
`timestamp`	TAI ISO 8601	International Atomic Time timestamp with nanosecond precision; timezone-independent
`payload_hash`	SHA3-512	Hash of the full event payload data structure

Field Name	Type	Description
		(serialized as canonical JSON); enables payload integrity verification without storing full payload in index
`pq_signature`	ML-DSA-87 bytes	Digital signature over all preceding fields (serialized in canonical order) by the originating node's ML-DSA-87 DID signing key
`anchor_hash`	SLH-DSA-SHA2-256s bytes	Long-lived notary signature over the record by the designated URIB anchor node; provides signature validity independent of ML-DSA key rotation
`merkle_root`	SHA3-512	Root of the Merkle tree for the batch settlement containing this record; enables individual record Merkle inclusion proof generation
`predecessor_hash`	SHA3-512	SHA3-512 hash of the immediately preceding URIB record in the append-only chain; provides tamper-evident chain integrity analogous to a blockchain

11.3 Settlement Batching

URIB events are batched per Chronos epoch. At each epoch boundary, all events generated during the epoch are collected, sorted by `timestamp`, and arranged as leaves of a binary Merkle tree. The Merkle root is computed and becomes the `merkle_root` field for all records in the batch. The batch anchor is signed by ≥ 2/3 of URIB anchor nodes using their SLH-DSA-SHA2-256s keys, providing a long-lived, quantum-resistant attestation of the batch's integrity that remains valid across all future PQ algorithm transitions. The batch commitment is itself recorded as a SETTLEMENT event type in URIB, closing the epoch record loop.

11.4 Audit Query Interface

The URIB exposes a DID-authenticated, read-only query API supporting the following query types: (1) event range queries by epoch ID range or timestamp range; (2) node-specific audit trails by node DID; (3) phase transition proofs returning all PHASE_TRANSITION events with associated governance DID signatures; (4) Merkle inclusion proofs for individual records, enabling any third party to verify that a specific URIB record is included in a committed batch without access to the full URIB; (5) predecessor hash chain traversal for contiguous chain integrity verification. All query sessions are authenticated by the querying party's DID signature (ML-DSA-87) and the session is logged as a query-log entry per the access policy defined in Section 10.3.

11.5 Retention Policy

URIB records are retained indefinitely on primary storage in append-only mode; no deletion or modification operation is supported by the URIB storage engine. Records older than 5 years are archived to cold storage (write-once optical media or equivalent) while maintaining full queryability via the URIB query interface. Cold storage integrity is verified annually by computing the Merkle root of the archived batch and comparing against the SLH-DSA-SHA2-256s-anchored batch commitment stored in the primary URIB index. Any cold storage integrity failure triggers an Aegis Tier 3 alert and mandatory incident investigation.

11.6 Air-Gapped URIB

Air-gapped nodes maintain a **local URIB shard** — a complete, cryptographically valid URIB chain rooted at the last shard synchronization point. The local shard operates identically to the master URIB for all local operations during the disconnection period. Upon reconnection (or physical media exchange), the local shard delta (all records generated since last synchronization) is exported and submitted to the master URIB merge process. Merge protocol: (1) records are sorted by `timestamp`; (2) duplicate records detected via `urib_id` uniqueness constraint and discarded; (3) ordering conflicts (records from different shards with overlapping epoch IDs) are resolved by Chronos epoch ordering precedence; (4) merged records are re-anchored with master URIB anchor node signatures and integrated into the Merkle batch for the current epoch. The local shard chain integrity is verified against

the local predecessor hash chain before merge submission; any chain integrity failure is reported as an Aegis Tier 2 event on the master node.

Section 12 — Runtime Enforcement Layer (REL)

12.1 Purpose

The **Runtime Enforcement Layer (REL)** provides kernel-level and application-level enforcement of the active cryptographic policy, preventing the invocation of deprecated or unauthorized algorithms at runtime regardless of the requesting application's configuration. The REL is the final enforcement backstop of the QUM: even if a misconfigured application or malicious code attempts to invoke a classical primitive in ENFORCING or STRICT mode, the REL blocks the operation before any classical cryptographic computation is performed.

12.2 Enforcement Architecture

- **Kernel Module — JasperSecMod:** A Jasper OS kernel security module that intercepts all cryptographic syscalls (key generation, encapsulation, signing, verification, encryption, decryption) via the kernel security hook framework. For each intercepted call, JasperSecMod: (a) identifies the requested algorithm via syscall parameter inspection; (b) validates the algorithm against the current DID Policy Anchor whitelist loaded into the kernel security database; (c) permits, blocks, or logs the operation per the active enforcement mode; (d) asynchronously logs the enforcement decision to URIB via a non-blocking ring buffer; (e) updates the Chronos epoch health score metric.
- **Application-Level Shim — JasperCryptAPI v3.0:** A mandatory application-level API shim that all Jasper OS applications must use for all cryptographic operations. JasperCryptAPI v3.0 provides a policy-aware wrapper around the PQ-Native Cryptographic Primitive Library. Applications

call JasperCryptAPI endpoints (e.g., `jasper_kem_encapsulate()`, `jasper_sign()`); the API internally resolves the appropriate PQ primitive per the current AAM state and Policy Anchor. Direct invocation of classical cryptographic libraries bypassing JasperCryptAPI is detectable via JasperSecMod and treated as a STRICT mode violation.

- **Hardware Enforcement — HSM Algorithm Whitelist:** The PQ-Native HSM firmware enforces an algorithm whitelist at the hardware level. Any request to the HSM for a classical algorithm operation (RSA key generation, ECC scalar multiplication, etc.) is rejected by the HSM firmware with a hardware-level error code, independent of JasperSecMod state. This provides a defense-in-depth enforcement layer that cannot be bypassed by kernel-level compromise.

12.3 Enforcement Modes

Mode	Description	Active Phase
PERMISSIVE	Classical primitive invocations are permitted but logged to URIB as policy advisory events. All invocations generate a Tier 1 Aegis advisory. PQ primitive invocations are preferred and recorded. No operations blocked. Used to establish telemetry baseline and ensure zero service disruption during HYBRID_V1.	HYBRID_V1
ENFORCING	Classical primitive invocations are blocked at JasperSecMod kernel level. Blocked invocations return `EKEYREJECTED` error code and generate a `SecurityException` at the application level. Each blocked invocation logged to URIB as an	PQ_ONLY

Mode	Description	Active Phase
	AEGIS_ALERT event and triggers Aegis Tier 2 minimum. Repeated violations (≥ 3 per epoch) escalate to Tier 3. All PQ primitive invocations proceed normally.	
STRICT	All non-whitelisted PQ algorithm variants blocked in addition to all classical primitives. Only the exact PQ primitive set specified in the current DID Policy Anchor whitelist is permitted. Policy whitelist is immutable until the next DID Governance-authorized Policy Anchor update. Any whitelist deviation blocked and logged. No runtime exceptions without DID Governance quorum vote and URIB record creation.	PQ_NATIVE

12.4 Exception Handling

No runtime exceptions to ENFORCING or STRICT mode are permitted without explicit DID Governance authorization. An emergency exception request must: (1) be submitted as a DID Governance vote by an authorized governance DID holder; (2) achieve the quorum threshold for algorithm approval (≥ 2/3); (3) specify an exact time-limited scope (maximum 1 epoch duration); (4) create a POLICY_CHANGE URIB record with all governance signatures attached; (5) trigger an Aegis Tier 2 alert logged as a policy deviation. JasperSecMod automatically re-enforces the full policy at the next epoch boundary regardless of whether the exception was consumed.

12.5 Observability

All REL enforcement events are exported to three observability channels: (1) the Chronos epoch health score, where each blocked invocation reduces the health

score by a configurable decrement (default: −0.01 per ENFORCING block; −0.05 per STRICT block); (2) the RLE telemetry bus, where aggregate enforcement metrics per node class per epoch are provided as training signals influencing the RLE's algorithm recommendation confidence model; (3) the URIB, where each enforcement event is asynchronously settled as part of the epoch batch. The aggregate REL enforcement dashboard is accessible via the DID Governance query interface per the access control policy defined in Section 10.3.

Section 13 — Acceptance Criteria Validator (ACV)

13.1 Purpose

The **Acceptance Criteria Validator (ACV)** is an automated, deterministic compliance gating system that constitutes the formal acceptance gate for all phase transitions, AGUP bundle applications, and major policy changes within the Jasper OS QUM. The ACV ensures that no upgrade action proceeds unless all relevant correctness, performance, integration, governance, integrity, and operational criteria are met. ACV outcomes are cryptographically signed and settled in URIB, providing a verifiable compliance audit trail for patent filing and regulatory purposes.

13.2 ACV Test Suite Structure

Category A — Cryptographic Correctness

#	Test	Pass Criterion
A-1	ML-KEM encapsulation/decapsulation round-trip	10,000 iterations; zero decapsulation failures; shared secret bit-equality verified
A-2	ML-DSA sign/verify round-trip	10,000 iterations; zero verification failures; signature size within FIPS 204 specification

#	Test	Pass Criterion
A-3	SLH-DSA sign/verify for long-lived certificate anchors	1,000 iterations; zero verification failures; signature determinism verified (same message, same key → identical signature)
A-4	Known-Answer Tests (KATs) per NIST FIPS 203/204/205 test vectors	100% pass on all published NIST KAT vectors for ML-KEM-768, ML-KEM-1024, ML-DSA-65, ML-DSA-87, SLH-DSA-SHA2-256s
A-5	Classical primitive rejection tests	In ENFORCING and STRICT modes: 100% of RSA, ECC, DSA, DH invocations return `EKEYREJECTED`; zero classical operations complete
A-6	Dual-signature verification (HYBRID_V1 only)	Payloads with only classical signature: rejected. Payloads with only PQ signature: rejected. Both required for acceptance.

Category B — Performance

#	Test	Pass Criterion
B-1	ML-KEM-1024 encapsulation latency (reference hardware)	≤ 2ms at p99 measured over 10,000 operations on reference server hardware
B-2	ML-DSA-87 signing latency (reference hardware)	≤ 5ms at p99 measured over 10,000 operations
B-3	Key rotation completion time (cluster)	≤ 1 Chronos epoch for all nodes in the managed cluster to complete key rotation
B-4	AGUP bundle application latency (Stages 5–6)	Complete within 1 local Chronos epoch for the target node class

#	Test	Pass Criterion
B-5	TLS 1.3 hybrid handshake overhead (HYBRID_V1)	≤ 15% additional latency at p95 relative to classical-only TLS 1.3 baseline

Category C — Aegis Integration

#	Test	Pass Criterion
C-1	Artificial fault injection: Tier 1–4 response trigger correctness	All four Tier response sequences execute correctly and completely for artificial fault injections simulating each trigger condition
C-2	Atomic rollback to last-known-good state	Rollback completes within 2 Chronos epochs of trigger; state hash matches pre-application snapshot; URIB ROLLBACK record created
C-3	Entropy depletion simulation	TRNG fallback to DRBG conditioner activates correctly; Aegis Tier 2 alert generated; no cryptographic operation permitted with sub-threshold entropy
C-4	Self-healing loop end-to-end validation	Full Aegis self-healing loop (detection → remediation → ACV → URIB → Chronos resume → RLE feedback) completes within 3 epochs for Tier 3 event

Category D — DID Governance

#	Test	Pass Criterion
D-1	Policy anchor refresh propagation	All connected nodes receive and validate updated Policy Anchor within 1 Chronos epoch of

#	Test	Pass Criterion
		DID Registry publication
D-2	Quorum vote simulation (2/3 and 1/2 thresholds)	Actions requiring ≥ 2/3 quorum rejected with n-1 votes present; accepted with exactly ≥ 2/3 votes; threshold boundary conditions verified
D-3	Key revocation propagation	Compromised key blocked across all connected nodes within 1 Chronos epoch of revocation publication
D-4	DID Document resolution correctness	All nodes resolve test DID Documents and extract correct ML-DSA-87 and ML-KEM-1024 public keys; resolution failure rate = 0%

Category E — URIB Integrity

#	Test	Pass Criterion
E-1	Merkle inclusion proof verification	1,000 randomly sampled URIB records verified via Merkle inclusion proof; 100% verification success rate
E-2	Predecessor hash chain integrity	Chain traversal over 10,000 consecutive URIB records shows 100% predecessor hash linkage correctness
E-3	Settlement latency	≤ 1 Chronos epoch from event generation to anchored URIB settlement (SLH-DSA anchor signature confirmed)
E-4	Tamper detection	Modification of any field in any URIB test record detectable by chain integrity verification within 1 epoch

Category F — Air-Gapped Operations

#	Test	Pass Criterion
F-1	Full AGUP cycle on isolated test node	Complete bundle compile → signing → physical transfer simulation → ingestion → verification → staged application → post-ACV → URIB shard export cycle completes successfully on isolated test node
F-2	Offline DID policy cache staleness handling	Node with DID policy cache > 5 epochs old generates Aegis Tier 1 advisory; operations continue with cached policy; cache refresh forced on next AGUP bundle delivery
F-3	Local URIB shard merge correctness	After simulated 30-day disconnection, local shard merge into master URIB completes with zero ordering conflicts, zero duplicate records, 100% predecessor hash chain integrity post-merge

13.3 ACV Gate Outcomes

Outcome	Condition	Consequence
PASS	All six categories (A–F) pass all tests with no exceptions	Phase transition or AGUP application proceeds as scheduled; URIB SETTLEMENT record created with PASS attestation
CONDITIONAL	Minor failures in Category B (performance) only; all other categories PASS; specific thresholds	RLE notified with performance telemetry; human review via DID Governance dashboard

Outcome	Condition	Consequence
	missed by ≤ 20%	required for authorization to proceed; URIB record created with CONDITIONAL status and reviewer DID required
FAIL	Any failure in Category A (cryptographic correctness), C (Aegis), D (DID governance), E (URIB integrity), or F (air-gapped operations); or Category B failures exceeding the CONDITIONAL threshold	Phase transition or AGUP application blocked; Aegis Tier 3 alert triggered; URIB AEGIS_ALERT record created with ACV failure payload; RLE receives negative training signal; remediation required before re-submission

---

Section 14 — Claims (Patent-Ready Format)

PATENT CLAIM NOTICE

The following claims are presented in draft patent claim format for review by patent counsel. Claims are not finalized until reviewed and approved by the designated Patent Counsel identified in Section 18. Claims are to be interpreted in light of the full specification set forth in Sections 1–13 of this document.

Independent Claims

Claim 1. A computer-implemented method for quantum-safe cryptographic migration of an operating system comprising: executing a first migration phase, designated HYBRID_V1, wherein a first set of classical cryptographic primitives and a first set of post-quantum cryptographic primitives are operated concurrently on at least one computing node, including performing

key encapsulation by executing both a classical key encapsulation operation and a post-quantum Module-Lattice-Based Key Encapsulation Mechanism (ML-KEM) operation and combining the resulting shared secrets via a key derivation function; evaluating, by a Recursive Learning Engine, a set of telemetry signals comprising latency metrics, error rates, and anomaly scores associated with the post-quantum primitives, wherein the Recursive Learning Engine applies a federated, privacy-preserving machine learning model with gradient aggregation via Secure Multi-Party Computation; responsive to the Recursive Learning Engine producing a confidence score of at least a first threshold for the post-quantum primitives and to a quorum-authorized policy change from a Decentralized Identity Governance Layer, transitioning to a second migration phase, designated PQ_ONLY, wherein the classical cryptographic primitives are blacklisted and only post-quantum primitives are permitted, with blacklist enforcement performed by a kernel-level security module; further transitioning, responsive to all exit criteria of the second migration phase being satisfied and a second quorum-authorized policy change, to a third migration phase, designated PQ_NATIVE, wherein classical cryptographic primitives are entirely removed from the operating system codebase and hardware configuration, and wherein post-quantum cryptographic primitives constitute the sole native cryptographic substrate; and delivering cryptographic upgrade artifacts to at least one air-gapped computing node without network connectivity by compiling, signing with at least three independent Decentralized Identifier authorized keys, and physically transferring a deterministic upgrade bundle verifiable by Merkle root comparison, wherein an identical bundle applied to an identical prior node state produces an identical post-application state.

Claim 2. A system for post-quantum cryptographic governance comprising: a Decentralized Identity Registry storing Decentralized Identifier documents each comprising at least one Module-Lattice-Based Digital Signature Algorithm (ML-DSA) verification key and at least one Module-Lattice-Based Key Encapsulation Mechanism (ML-KEM) key agreement key, wherein the Decentralized Identifier documents conform to the W3C Decentralized Identifier Core Specification; a governance module configured to authorize

cryptographic policy changes, phase transitions, and emergency actions only upon receipt of digitally signed approvals from a required quorum of authorized Decentralized Identifier holders, wherein each approval is signed with an ML-DSA signing key; a Chronos Heartbeat Orchestration Engine configured to manage cryptographic upgrade lifecycle events through deterministic, epoch-driven scheduling, wherein each epoch is assigned a monotonically increasing identifier, cryptographically signed by a master signing key, and anchored in an append-only audit ledger; and a Universal Resource Identity Bundle comprising an immutable, Merkle-chained sequence of cryptographic event records, wherein each record comprises a time-ordered unique identifier, an event type, an originating node Decentralized Identifier, an epoch identifier, a timestamp, a hash of event payload, a post-quantum digital signature by the originating node, a long-lived stateless hash-based signature anchor, a Merkle root of the batch settlement containing the record, and a hash of the immediately preceding record in the chain.

Claim 3. A non-transitory computer-readable medium storing instructions that, when executed by at least one processor, cause the processor to: enforce, via a kernel security module intercepting cryptographic system calls, a cryptographic algorithm policy derived from a currently active Decentralized Identifier-anchored Policy Anchor document, wherein enforcement operates in one of: a first mode permitting classical algorithm invocations while logging them; a second mode blocking all classical algorithm invocations and returning a security exception; and a third mode blocking all algorithm invocations not explicitly whitelisted in the Policy Anchor; detect cryptographic failures, policy deviations, entropy source degradation, and upgrade anomalies via a self-healing subsystem operating a four-tier alert response hierarchy; responsive to detection of a tier-three or tier-four alert condition, atomically restore a computing node to a cryptographically signed prior-state snapshot preserved prior to the most recent upgrade operation; and record all enforcement events, anomaly alerts, and rollback operations as cryptographically signed, Merkle-chained entries in the Universal Resource Identity Bundle audit ledger.

Dependent Claims

Claim 4. The method of Claim 1, wherein the first migration phase further comprises: performing digital signature operations by computing both a classical digital signature and a post-quantum Module-Lattice-Based Digital Signature Algorithm (ML-DSA) signature over each payload; and accepting a signed payload only when both the classical digital signature and the post-quantum digital signature independently verify as valid.

Claim 5. The method of Claim 1, wherein the second migration phase further comprises: enforcing a classical primitive blacklist at a hardware security module firmware level, wherein any invocation of a classical cryptographic algorithm is rejected by hardware independent of operating system software state; rotating post-quantum key encapsulation keypairs at an interval of a first configurable number of heartbeat epochs; and rotating post-quantum signing keypairs at an interval of a second configurable number of heartbeat epochs greater than the first configurable number.

Claim 6. The method of Claim 1, wherein the third migration phase further comprises: verifying operating system boot integrity using a stateless hash-based digital signature algorithm (SLH-DSA) root certificate anchoring the full boot chain Merkle hash; executing constant-time implementations of all post-quantum cryptographic primitives with cache-flush operations between sequential cryptographic operations; and monitoring, by the self-healing subsystem, timing variance of cryptographic operations for deviation from constant-time profiles exceeding a configurable statistical threshold.

Claim 7. The system of Claim 2, wherein governance actions requiring a two-thirds quorum of authorized Decentralized Identifier holders comprise: algorithm approvals for new post-quantum primitives or security parameter variants; phase transition authorizations; Decentralized Identifier key revocation and replacement; and emergency rollback from the second migration phase to the first migration phase,

wherein such emergency rollback further requires a concurrent tier-four self-healing subsystem alert affecting a threshold proportion of the node population.

Claim 8. The system of Claim 2, wherein the Universal Resource Identity Bundle further comprises: a batch settlement mechanism wherein all event records generated within a single Chronos heartbeat epoch are organized as leaves of a binary Merkle tree, the Merkle root is computed and embedded in each record's Merkle root field, and the batch is anchored by at least two-thirds of designated Universal Resource Identity Bundle anchor nodes using stateless hash-based digital signatures providing long-lived quantum-resistant attestation.

Claim 9. The system of Claim 2, wherein the Chronos Heartbeat Orchestration Engine implements Byzantine-fault-tolerant distributed epoch consensus requiring at least 2f+1 agreeing nodes in a cluster of at least 3f+1 total nodes for any phase-transition epoch to be confirmed, wherein no phase-transition action is executed on any cluster node until the consensus threshold is achieved.

Claim 10. The non-transitory computer-readable medium of Claim 3, wherein the four-tier alert response hierarchy comprises: a first advisory tier responsive to minor performance deviations, wherein the self-healing subsystem logs the event and notifies the Recursive Learning Engine without operational change; a second warning tier responsive to policy drift or entropy degradation, wherein the self-healing subsystem re-anchors the node to the current Policy Anchor and activates entropy fallback mechanisms; a third critical tier responsive to key material corruption or upgrade integrity failure, wherein the self-healing subsystem isolates the affected node, initiates atomic rollback, and creates a rollback audit record; and a fourth emergency tier responsive to systemic post-quantum primitive failure, wherein the self-healing subsystem initiates full node quarantine, suspends all cryptographic operations, and notifies all authorized Decentralized Identifier governance holders.

Claim 11. The non-transitory computer-readable medium of Claim 3, wherein atomic rollback comprises: preserving a cryptographically signed snapshot of the complete node cryptographic state prior to each upgrade operation, the snapshot comprising active key fingerprints, algorithm agility matrix state hash, runtime enforcement layer mode, Chronos epoch position, Policy Anchor hash, and cryptographic primitive library version hash; and restoring the node to the snapshot state in a single atomic transaction upon rollback trigger, followed by execution of cryptographic correctness and self-healing integration acceptance criteria validation tests before resuming normal epoch participation.

Claim 12. The non-transitory computer-readable medium of Claim 3, further comprising instructions to execute a six-category acceptance criteria validation suite prior to any phase transition or upgrade bundle application, comprising: cryptographic correctness tests including round-trip encapsulation and signature tests and known-answer tests per applicable Federal Information Processing Standards; performance latency tests against configurable thresholds; self-healing subsystem integration tests including fault injection and rollback validation; Decentralized Identity governance tests including policy propagation and quorum threshold verification; Universal Resource Identity Bundle integrity tests including Merkle inclusion proof verification and chain integrity verification; and air-gapped operation tests including full upgrade pipeline cycle and local shard merge correctness validation.

Claim 13. The method of Claim 1 and the system of Claim 2, wherein the Recursive Learning Engine, operating as part of the system, performs federated machine learning by: training a local model replica on telemetry data generated exclusively on each respective computing node without transmitting raw telemetry outside the node; aggregating gradient updates from a plurality of computing nodes via a Secure Multi-Party Computation protocol that prevents any single aggregating party from observing individual node gradient updates; applying Byzantine-fault-tolerant gradient aggregation that excludes gradient contributions from nodes whose updates deviate from the population distribution by more than a configurable statistical threshold; and producing algorithm recommendation vectors with associated confidence scores, wherein recommendations with confidence scores below a configurable threshold are

withheld from automatic application and submitted for human review via the Decentralized Identity Governance Layer.

Claim 14. The method of Claim 13, wherein for at least one computing node operating without network connectivity, the Recursive Learning Engine operates in an air-gapped mode comprising: receiving updated global model weights as a signed artifact within an Air-Gapped Upgrade Pipeline bundle delivered via encrypted physical media; performing local model inference exclusively against telemetry data generated on the air-gapped node; generating local algorithm recommendations without participating in the Secure Multi-Party Computation gradient aggregation protocol; batching locally generated telemetry for export via secure physical media transfer upon the next available physical connection or media exchange event; and incorporating the exported telemetry into the next Secure Multi-Party Computation aggregation round upon reconnection, after verification of telemetry data integrity via the originating node's Decentralized Identifier signing key.

Section 15 — Implementation Roadmap

Phase	Timeline	Key Milestones	Dependencies	Success Metrics
Phase 0: Foundation	Q3 2026	FIPS 140-3 Level 3 HSM procurement and validation with PQ-Native firmware readiness JasperCryptAPI v3.0 development and internal QA (all PQ primitive bindings,	HSM vendor delivery (12-week lead); NIST FIPS 203/204/205 reference implementation libraries; JasperCryptAPI v3.0 internal review	JasperCryptAPI v3.0 Unit Test coverage ≥ 95%; DID Registry uptime ≥ 99.9% over 30-day pilot; URIB write latency ≤ 100ms at p99

Phase	Timeline	Key Milestones	Dependencies	Success Metrics
		PERMISSIVE mode REL) DID Registry deployment (3-node Byzantine consensus pilot cluster) `did:jasper` method specification finalization and W3C registration submission URIB storage engine deployment (primary + cold archive configuration) Chronos Heartbeat Engine v1.0 deployment (single-node epoch driver) ACV test suite v1.0 completion (Categories A and B)		
Phase 1: HYBRID_V1 Rollout	Q4 2026 – Q1 2027	Dual-stack cryptographic library deployment across all connected server-class nodes TLS 1.3 hybrid key exchange (ECDH + ML-KEM-768) enabled on all TLS endpoints Chronos epoch calibration	Phase 0 completion; all target nodes upgraded to JasperCryptAPI v3.0; DID Governance quorum established (minimum 5 authorized DID holders)	HYBRID_V1 exit criteria met per Section 5.1.4; RLE confidence score ≥ 0.92; zero Tier 3/4 Aegis alerts for 30 consecutive epochs; 100% URIB settlement coverage

Phase	Timeline	Key Milestones	Dependencies	Success Metrics
		(epoch duration tuning per node class under production load) URIB bootstrap — first epoch anchored; AGUP initial bundle pipeline operational RLE v1.0 deployed; federated model training initiated across pilot node set Aegis PERMISSIVE mode monitoring active; baseline telemetry established ACV full suite (Categories A–F) deployed and validated First air-gapped node AGUP bundle cycle completed		
Phase 2: PQ_ONLY Migration	Q2 – Q3 2027	Classical primitive deprecation — RSA, ECC blacklist activated in REL ENFORCING mode Full Aegis activation — all five	HYBRID_V1 exit criteria fully attested in URIB; DID Governance Policy Anchor updated to PQ_ONLY; HSM firmware upgraded to enforce classical	PQ_ONLY exit criteria met per Section 5.2.4; RLE confidence score ≥ 0.97; zero classical primitive invocations in URIB for 60 consecutive epochs; ACV

Phase	Timeline	Key Milestones	Dependencies	Success Metrics
		detection mechanisms active All X.509 certificates reissued as pure PQ (ML-DSA-87 signed) AGUP full rollout to all air-gapped node classes; SPMTP operational RLE v2.0 with full Byzantine-fault-tolerant SMPC deployed Chronos multi-node PBFT epoch consensus activated (full node population) URIB audit query interface — external patent counsel access provisioned Security audit by external cryptographic firm (FIPS 140-3 validation scope)	blacklist at hardware level	PASS all categories all nodes
Phase 3: PQ_NATIVE Steady State	Q4 2027 onward	HSM firmware finalization — classical accelerators disabled via OTP fuse; PQ-Native firmware	PQ_ONLY exit criteria fully attested in URIB; DID Governance Policy Anchor updated to PQ_NATIVE;	REL STRICT mode violation rate = 0 per epoch; Aegis Tier 3/4 events = 0 per 30-day

Phase	Timeline	Key Milestones	Dependencies	Success Metrics
		write-locked REL STRICT mode activated across all nodes JasperCryptAPI v3.0 PQ-Native build — all classical API endpoints removed from source tree PQ-signed bootloader (ML-DSA-87) deployed; SLH-DSA Secure Boot root activated Ongoing RLE optimization — NIST PQC Round 5+ candidate monitoring initiated Annual URIB cold archive integrity verification procedure established Lightweight CIP amendment process for future PQ algorithm variant adoption activated	FIPS 140-3 HSM PQ-Native validation certificate obtained	rolling window; RLE migration phase readiness score ≥ 0.99 for PQ_NATIVE; annual ACV Category A–F PASS confirmed

Section 16 — Security Analysis

16.1 Threat Model

The Jasper OS QUM is designed to protect against the following adversary classes:

Adversary Class	Capabilities	Assets Targeted
Quantum-Capable Adversary	Access to a cryptographically relevant quantum computer capable of executing Shor's algorithm against RSA-4096 and ECC-P384 in practical time; Grover's algorithm capability against symmetric primitives	Classical key material; historical ciphertext captured via HNDL attacks; certificate integrity
Insider Threat	Authorized access to governance systems; ability to submit malicious policy updates or governance votes; potential access to signing keys	DID Governance policy anchors; URIB records; upgrade bundle signing keys
Supply Chain Attacker	Ability to inject malicious firmware or library artifacts into the upgrade supply chain; compromise of build infrastructure	AGUP bundles; PQ primitive library implementations; HSM firmware
Network Attacker	Active man-in-the-middle on network channels; ability to intercept and replay cryptographic messages; DDoS against governance infrastructure	TLS handshakes; DID Registry communications; Chronos epoch synchronization

16.2 Quantum Resistance

The QUM's algorithm selections provide the following quantum resistance profile:

- **Shor's Algorithm (RSA, ECC elimination):** ML-KEM and ML-DSA are founded on Module Learning With Errors (MLWE) and Module Short Integer

Solution (MSIS) hardness assumptions, which are not known to be vulnerable to Shor's algorithm or any known quantum attack. Their security levels are defined against quantum adversaries: ML-KEM-1024 targets NIST PQC Security Level 5 (≥ 256-bit quantum security); ML-DSA-87 targets Security Level 5 equivalently.

- **Grover's Algorithm (symmetric primitive impact):** Grover's algorithm provides a quadratic speedup against symmetric cryptography, effectively halving the security level in terms of bit strength. The QUM's use of SHA3-512 for all hashing (providing 256-bit quantum security) and HKDF for key derivation with 256-bit session keys ensures that Grover's algorithm does not reduce symmetric security below the 128-bit minimum recommended threshold.
- **SLH-DSA (long-lived signature quantum resistance):** SLH-DSA is a hash-based signature scheme whose security is reducible to the one-wayness of the underlying hash function. Against quantum adversaries applying Grover's algorithm, the SHA2-256s variant retains sufficient security margin at the chosen parameter set.

16.3 Supply Chain Security

AGUP bundles are protected against supply chain attacks by: (1) triple-signing by three independent DID-authorized governance keys, requiring simultaneous compromise of three distinct key holders for a malicious bundle to pass verification; (2) Merkle tree signing of all bundle components, ensuring that the modification of any single component invalidates the Merkle root comparison; (3) SBOM generation per NTIA minimum elements with SLH-DSA-SHA2-256s signature, providing a long-lived, tamper-evident component inventory; (4) node-specific ML-KEM-1024 media encryption, ensuring that a captured upgrade bundle cannot be decrypted by any node other than the intended recipient; and (5) ACV Category A pre-flight execution before any bundle component is applied, providing a final integrity gate at the point of application.

16.4 Governance Attack Surface

The DID Governance Layer's quorum requirements provide structural resistance to governance attacks: a ≥ 2/3 quorum requirement for all high-impact actions means that an attacker must simultaneously compromise at least one-third plus one of all authorized governance DID holders to achieve unauthorized policy changes. DID key rotation limits the exposure window of any individual compromised key to the period between compromise and detection/revocation. The URIB provides a tamper-evident record of all governance actions, enabling post-hoc detection of unauthorized governance activity even if it occurred before detection. Emergency key compromise revocation, described in Section 10.5, provides a defined response procedure that minimizes the window of exposure.

16.5 Known Limitations and Mitigations

Known Limitation	Mitigation
Implementation side-channels in PQ primitive library (timing attacks, cache side-channels)	Mandatory constant-time implementation requirement for all PQ primitive library code; cache-flush routines between sequential cryptographic operations; Aegis timing variance monitoring with configurable deviation threshold; regular external security audit of constant-time correctness
Harvest-now-decrypt-later for data encrypted during HYBRID_V1 phase under classical-only paths	All new key exchange operations from HYBRID_V1 commencement use ML-KEM forward-secret ephemeral keys; session keys are never derived from classical-only paths after Phase 1 commencement. Legacy data encrypted before HYBRID_V1 requires re-encryption under PQ keys — a data re-encryption project governed by a separate CIP amendment
Physical security of AGUP transfer media (chain-of-custody risk)	Node-specific ML-KEM-1024 encryption ensures media is unreadable without the target node's HSM private key; chain-of-custody DID signatures create an auditable custody trail; physical media destruction after verified application is a required operational procedure

Known Limitation	Mitigation
SMPC computational overhead for RLE gradient aggregation at scale	SMPC aggregation scheduled at 10-epoch intervals rather than per-epoch; gradient compression techniques applied before SMPC input; air-gapped nodes excluded from SMPC with batched telemetry incorporation on reconnection
PQ algorithm standardization evolution (post-NIST Round 4 updates)	Algorithm Agility Matrix provides hot-swap capability for PQ primitive variants; lightweight CIP amendment process for new NIST-standardized algorithms; RLE monitoring of NIST PQC Round 5+ candidates

---

Section 17 — References and Standards Compliance

#	Reference
[1]	National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM).* U.S. Department of Commerce, August 2024.
[2]	National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).* U.S. Department of Commerce, August 2024.
[3]	National Institute of Standards and Technology. *FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA).* U.S. Department of Commerce, August 2024.
[4]	National Institute of Standards and Technology. *NIST Special Publication 800-90B: Recommendation for the Entropy Sources Used for Random Bit Generation.* U.S. Department of Commerce, January 2018.

#	Reference
[5]	World Wide Web Consortium (W3C). *Decentralized Identifiers (DIDs) v1.0: Core Architecture, Data Model, and Representations.* W3C Recommendation, July 2022. https://www.w3.org/TR/did-core/
[6]	National Telecommunications and Information Administration (NTIA). *The Minimum Elements For a Software Bill of Materials (SBOM).* U.S. Department of Commerce, July 2021.
[7]	Rescorla, E. *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3.* Internet Engineering Task Force, August 2018.
[8]	Stebila, D., Fluhrer, S., and Gueron, S. *IETF Internet-Draft: Hybrid Key Exchange in TLS 1.3 (draft-ietf-tls-hybrid-design).* Internet Engineering Task Force. [Work in Progress]
[9]	National Institute of Standards and Technology. *NIST Special Publication 800-208: Recommendation for Stateful Hash-Based Signature Schemes.* U.S. Department of Commerce, October 2020.
[10]	Institute of Electrical and Electronics Engineers. *IEEE Standard 2830-2021: Standard for Technical Framework and Requirements of Trusted Execution Environments Based Shared Machine Learning.* IEEE, 2021.
[11]	International Organization for Standardization / International Electrotechnical Commission. *ISO/IEC 27001:2022: Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements.* ISO/IEC, 2022.
[12]	National Institute of Standards and Technology. *FIPS 140-3: Security Requirements for Cryptographic Modules.* U.S. Department of Commerce, March 2019.
[13]	Castro, M. and Liskov, B. *Practical*

#	Reference
	Byzantine Fault Tolerance. Proceedings of the 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI), February 1999.
[14]	National Institute of Standards and Technology. *Post-Quantum Cryptography Standardization Project: Round 4 Submissions.* Computer Security Resource Center. https://csrc.nist.gov/projects/post-quantum-cryptography
[15]	Levin, R., et al. *RFC 9562: Universally Unique IDentifiers (UUIDs) — Version 7 (Time-Ordered).* Internet Engineering Task Force, May 2024.

Section 18 — Revision History and Approvals

18.1 Revision History

Version	Date	Author	Changes	DID-Authorized Approver
1.0.0	2026-07-19	Jasper OS Security Architecture Team	Initial master record. Supersedes HYBRID_V1 CIP, PQ_ONLY CIP, PQ_NATIVE CIP, Aegis CIP, and Chronos CIP. Incorporates all prior CIP content into unified JOSC-QUM-001	`did:jasper:josec-arch-001` ML-DSA-87 Sig: [PENDING]

Version	Date	Author	Changes	DID-Authorized Approver
			master specification. Added Section 14 (Patent Claims), Section 15 (Roadmap), Section 16 (Security Analysis). Formalized all subsystem specifications to patent-filing quality. ACV test suite Categories A–F fully specified. URIB schema formalized with UUID v7 and SLH-DSA anchor fields.	

18.2 Formal Approval Block

The undersigned hereby attest that this document — CIP JOSC-QUM-001, Version 1.0.0 — has been reviewed and approved in their respective areas of authority, and that the content is accurate, complete, and appropriate for patent filing submission.